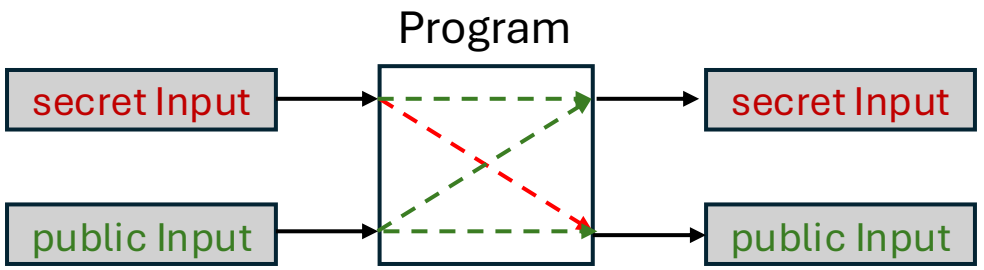
A Type System for Data Privacy Compliance in Active Object Languages

Chinmayi Baramashetru
(with Paola Giannini, Silvia Lizeth Tapia Tarifa & Olaf Owe)

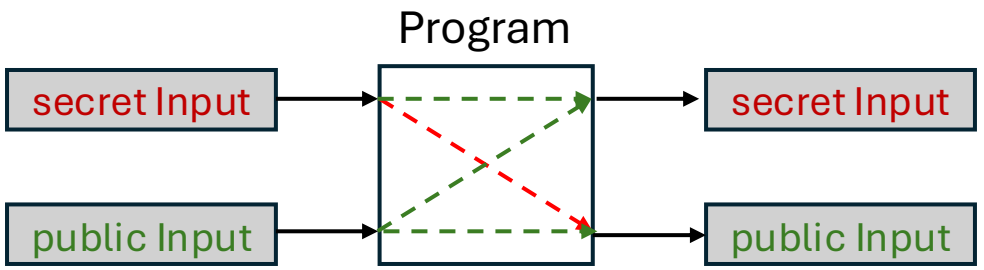
Information Flow

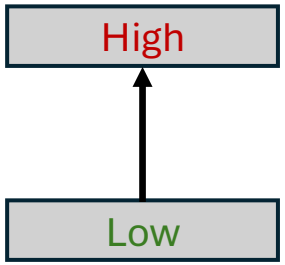
- Information flows through systems
- We want to both understand how this information flows, and possibly restrict it



Information Flow

- Information flows through systems
- We want to both understand how this information flows, and possibly restrict it

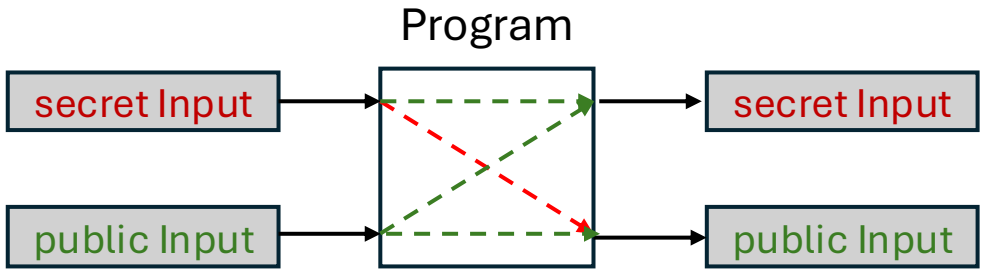


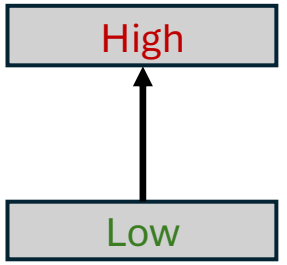


$L \sqsubseteq H$

Information Flow

- Information flows through systems
- We want to both understand how this information flows, and possibly restrict it



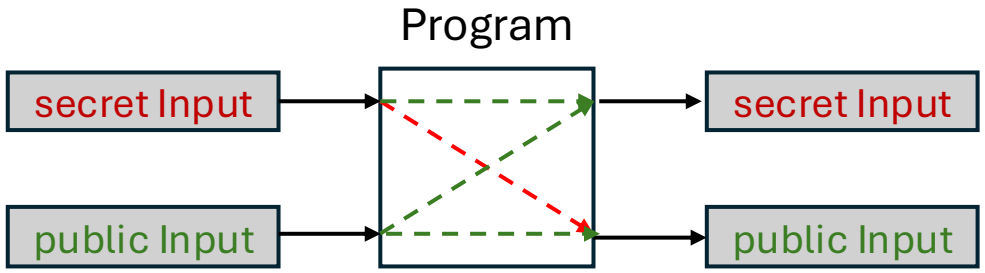


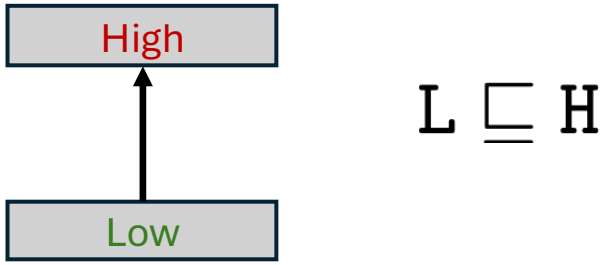
$L \sqsubseteq H$

```
High : Int secret = 25
Low  : Int public

public = secret
```

Information Flow

- Information flows through systems
- We want to both understand how this information flows, and possibly restrict it



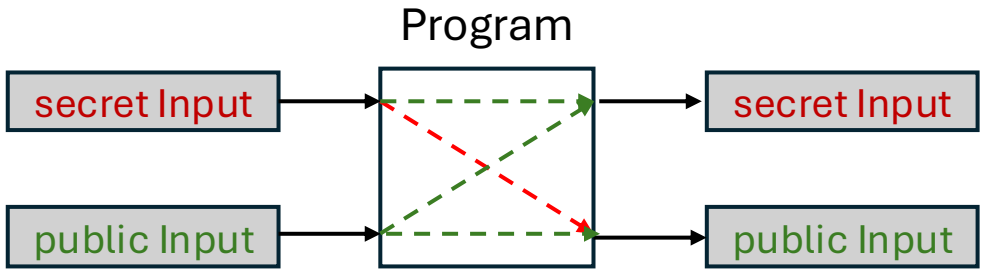


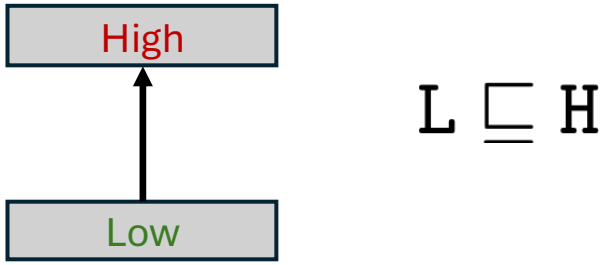

```
High : Int secret = 25
Low  : Int public

public = secret
```

```
High : Int secret;
Low : Int public = 0;

if (secret >0)
public = 1;
```

Information Flow

- Information flows through systems
- We want to both understand how this information flows, and possibly restrict it






```
High : Int secret = 25
Low  : Int public

public = secret
```

- Annotations helps tractability
- IFC stops both explicit and implicit flows
- Static analysis = Type checking

```
High : Int secret;
Low : Int public = 0;

if (secret >0)
public = 1;
```

Information Flow : Dependency

- Information flow characterizes **semantic dependencies** between program variables

Information Flow : Dependency

- Information flow characterizes **semantic dependencies** between program variables

```
high : diagnosis;  // sensitive data
high : stats;

stats = diagnosis; // use data for analytics
```

Information Flow : Dependency

- Information flow characterizes **semantic dependencies** between program variables

```
high : diagnosis;  // sensitive data
high : stats;

stats = diagnosis; // use data for analytics
```

```
stats depends on diagnosis
```

Information Flow : Dependency

- Information flow characterizes **semantic dependencies** between program variables

```
high : diagnosis;  // sensitive data
high : stats;

stats = diagnosis; // use data for analytics
```

```
stats depends on diagnosis
```

- Dependency tells us ***that influence exists***, not whether that influence is *permitted*

Privacy Policies and Consent Enforcement

- Security policies prevent unauthorized access : static labels, lattices, noninterference proofs

```
Only doctors can access medical records
```

- read/write, static, secrecy

Privacy Policies and Consent Enforcement

- Security policies prevent unauthorized access : Static labels, lattices, noninterference proofs

```
Only doctors can access medical records
```

- read/write, static, secrecy

- Privacy policies govern how data about people is handled

```
Patient data can be used for treatment but
not for marketing by doctors and nurses
```

- purposes & consent
- dynamic & revocable
- is this use legitimate?
- accountability

Privacy Policies and Consent Enforcement

- Security policies prevent unauthorized access : Static labels, lattices, noninterference proofs

```
Only doctors can access medical records
```

- read/write, static, secrecy

- Privacy policies govern how data about people is handled

```
Patient data can be used for treatment but
not for marketing by doctors and nurses
```

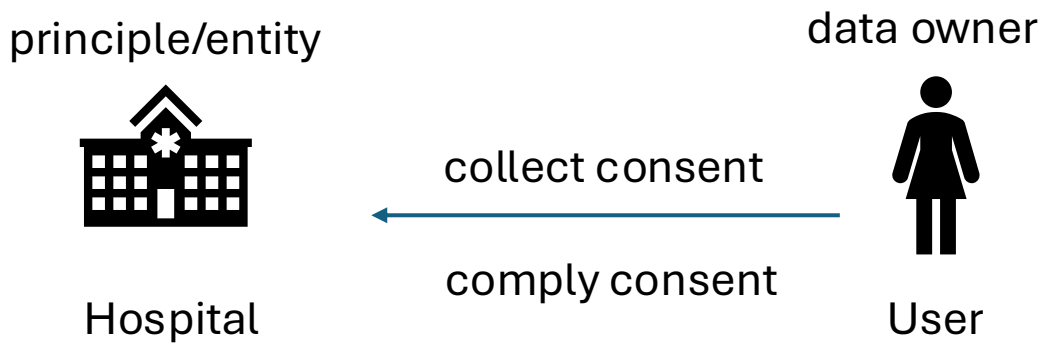
- purposes & consent
- dynamic & revocable
- is this use legitimate?
- accountability

Consent Enforcement

- Policy languages (P3P, EPAL)
- Usage Control (UCON)
- Modern GDPR-style consent (Revocable, purpose limited & time sensitive)

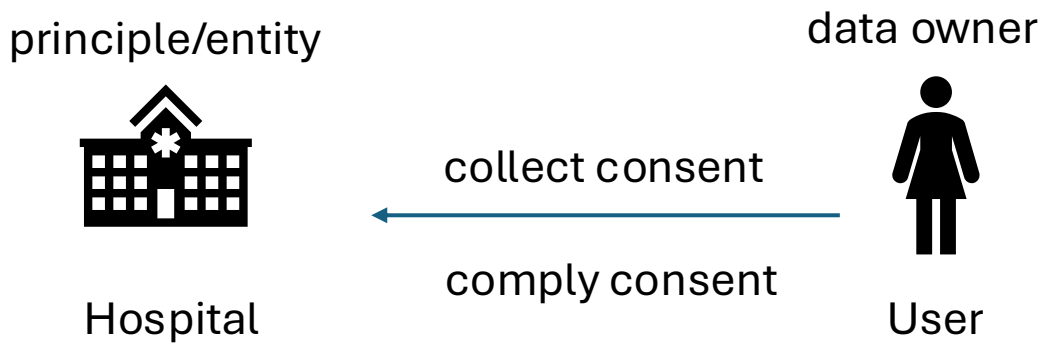
Policy Aware Dependency Tracking

- Purpose and consent-aware policies



Policy Aware Dependency Tracking

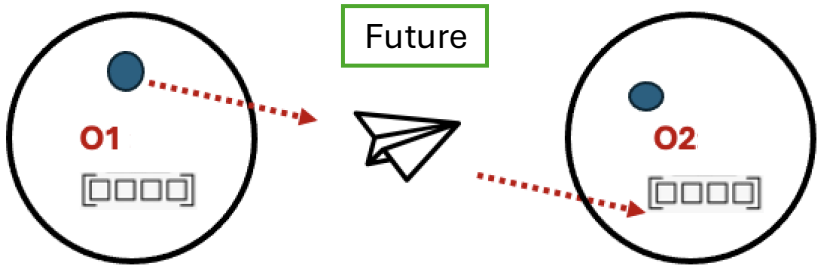
- Purpose and consent-aware policies



- Static tracking of policy-relevant dependencies
- Types derive policy obligations
- Consent is dynamic and revocable
- Runtime validation against dynamic consent

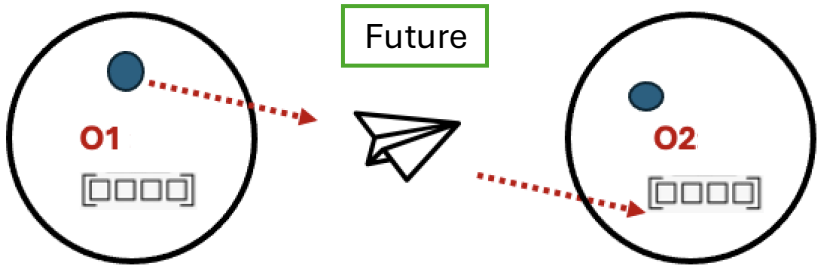
Active Object Languages

- Parallel entities that communicate via asynchronous messages and mailboxes



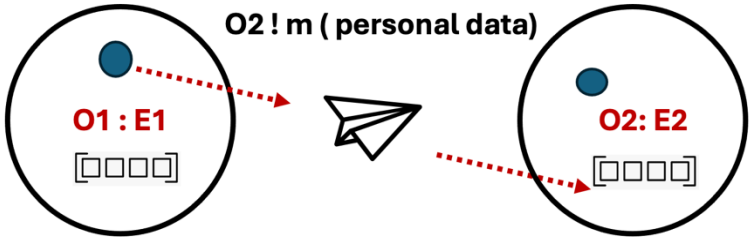
Active Object Languages

- Parallel entities that communicate via asynchronous messages and mailboxes

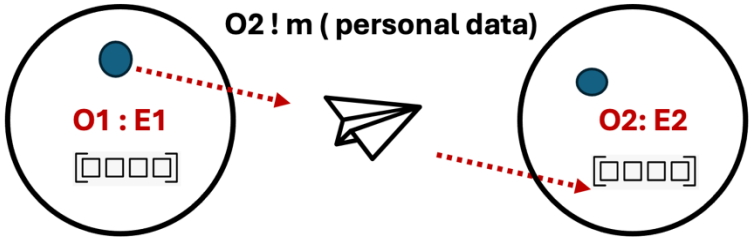


- **Encapsulation:** each object owns its state and executes independently without shared memory
- **Asynchronous calls:** method calls return futures (mailboxes)
- **Activation model:** objects process one queued request at a time
- **Explicit synchronization:** futures (get) define clear sync points

Our Approach

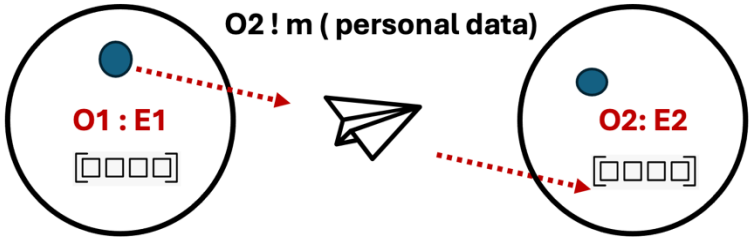


Our Approach

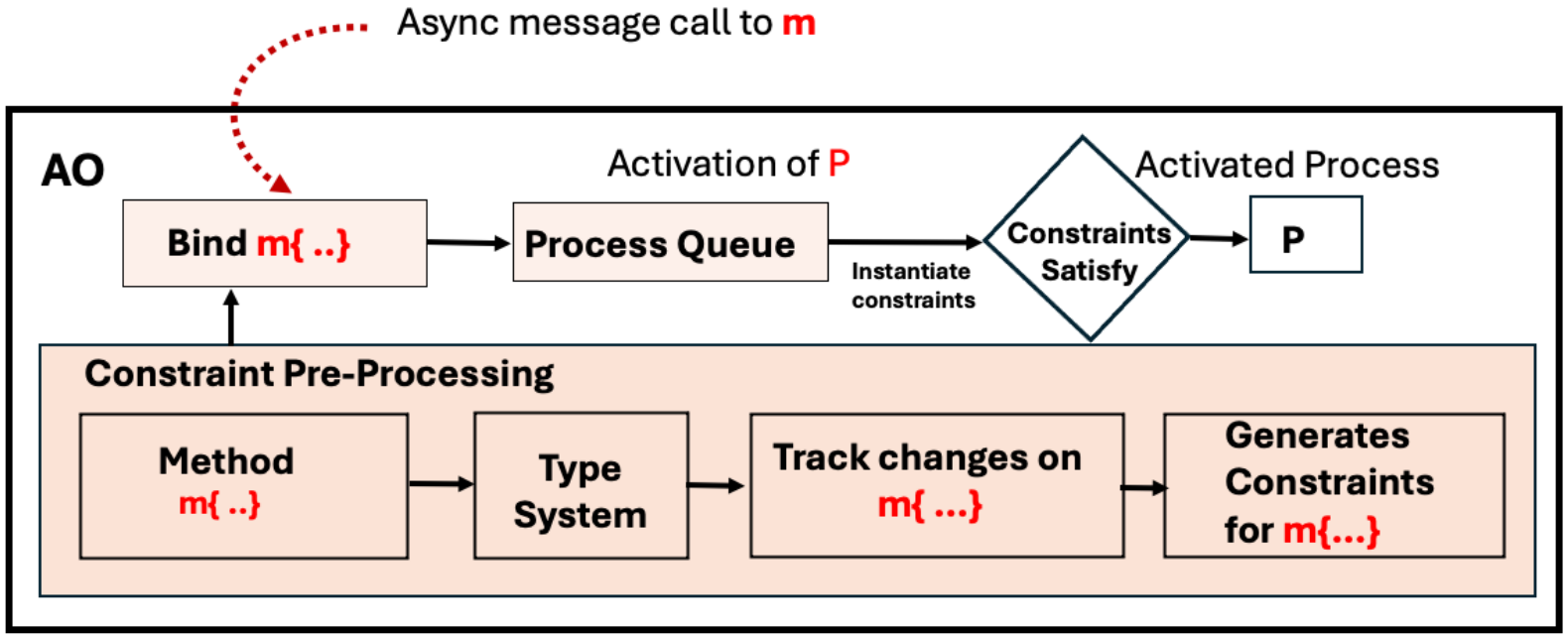


A dependency is legal only if the required actions are consented at runtime.

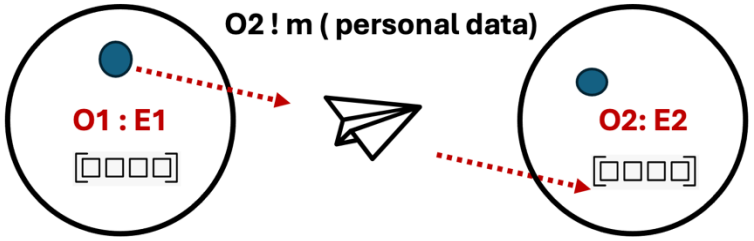
Our Approach



A dependency is legal only if the required actions are consented at runtime.

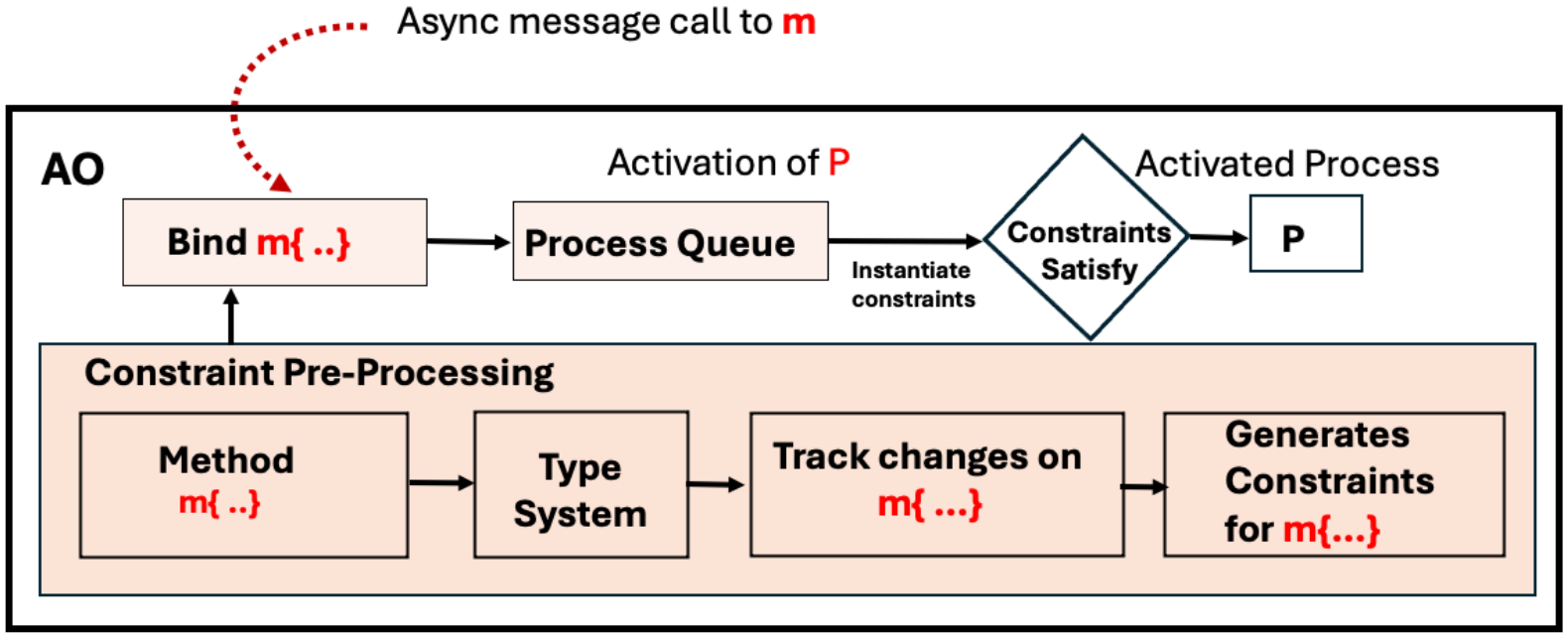


Our Approach



A dependency is legal only if the required actions are consented at runtime.

Invariant: Activated processes are policy-compliant by construction



Compile-time
- Track dependencies
- Generate constraints

Runtime
- Instantiate constraints
- Check at method activation

TyPAOL

$$
\begin{aligned}
PR &::= E\ P\ \overline{CL}\ \{\overline{A}\,\overline{x};\ sts\} \\
E &::= \textbf{entities}\ \widehat{\eta}; \\
P &::= \textbf{purposes}\ \widehat{p}; \\
A &::= B \mid C \mid \mathsf{U} \\
B &::= T \mid T^{\#} \\
T &::= Int \mid Bool \mid Unit \mid T * \cdots * T \mid \cdots \\
CL &::= \textbf{class}\ C(\overline{A}\,\overline{x})\ \{\ \overline{M}\ \} \\
M &::= B[:\Lambda]\ m(\overline{A}\,\overline{x})\ \{\ \overline{A}\,\overline{x};\ sts\ \} \\
\Lambda &::= (\widehat{vr}, t) \\
t &::= \langle \widehat{x}, \widehat{p} \rangle
\end{aligned}
$$

$$
\begin{aligned}
sts &::= \epsilon \mid s; sts \mid \textbf{return}\ \alpha \\
s &::= \textbf{skip} \mid x := rhs \mid \textbf{this}.x := \alpha \\
&\quad \mid \textbf{if}\ \alpha\ s\ \textbf{else}\ s \mid [\textbf{Fut}\langle B \rangle\ x :=]\ vr@m(\overline{\alpha}) \\
&\quad \mid \{sts\} \mid x.\textbf{addCon}(\rho) \mid x.\textbf{remCon}(\rho) \\
rhs &::= \alpha \mid \textbf{new}\ C(\overline{\alpha})\ \textbf{of}\ \eta \mid \textbf{new user} \mid x.\textbf{get} \\
@ &::= ! \mid !! \\
\alpha &::= e \mid e\ \textbf{tag}\ t \\
e &::= v \mid vr \mid (\overline{e}) \mid e.n \mid e\ op\ e \\
vr &::= x \mid \textbf{this}.x \mid \textbf{this} \\
\rho &::= (\widehat{\eta}, \widehat{a}, \widehat{p}) \\
a &::= \textbf{use} \mid \textbf{collect} \mid \textbf{store} \mid \textbf{transfer}
\end{aligned}
$$

TyPAOL

$$
\begin{aligned}
PR &::= E\ P\ \overline{CL}\ \{\overline{A}\,\overline{x};\ sts\} \\
E &::= \textbf{entities}\ \widehat{\eta}; \\
P &::= \textbf{purposes}\ \widehat{p}; \\
A &::= B \mid C \mid \mathsf{U} \\
B &::= T \mid T^{\#} \\
T &::= Int \mid Bool \mid Unit \mid T * \cdots * T \mid \cdots \\
CL &::= \textbf{class}\ C(\overline{A}\,\overline{x})\ \{\ \overline{M}\ \} \\
M &::= B[:\Lambda]\ m(\overline{A}\,\overline{x})\ \{\ \overline{A}\,\overline{x};\ sts\ \} \\
\Lambda &::= (\widehat{vr}, t) \\
t &::= \langle \widehat{x}, \widehat{p} \rangle
\end{aligned}
$$

$$
\begin{aligned}
sts &::= \epsilon \mid s; sts \mid \textbf{return}\ \alpha \\
s &::= \textbf{skip} \mid x := rhs \mid \textbf{this}.x := \alpha \\
&\quad \mid \textbf{if}\ \alpha\ s\ \textbf{else}\ s \mid [\textbf{Fut}\langle B\rangle\ x :=]\ vr@m(\overline{\alpha}) \\
&\quad \mid \{sts\} \mid x.\textbf{addCon}(\rho) \mid x.\textbf{remCon}(\rho) \\
rhs &::= \alpha \mid \textbf{new}\ C(\overline{\alpha})\ \textbf{of}\ \eta \mid \textbf{new user} \mid x.\textbf{get} \\
@ &::= ! \mid !! \\
\alpha &::= e \mid e\ \textbf{tag}\ t \\
e &::= v \mid vr \mid (\overline{e}) \mid e.n \mid e\ op\ e \\
vr &::= x \mid \textbf{this}.x \mid \textbf{this} \\
\rho &::= (\widehat{\eta}, \widehat{a}, \widehat{p}) \\
a &::= \textbf{use} \mid \textbf{collect} \mid \textbf{store} \mid \textbf{transfer}
\end{aligned}
$$

Types and Type Environments

Data Types

Tag annotations record provenance and explicit tags

$$\mathrm{DT} ::= T^{\Lambda} \qquad \Lambda ::= (\widehat{vr}, t) \mid \epsilon$$

Types and Type Environments

Data Types

Tag annotations record provenance and explicit tags

$$\mathrm{DT} ::= T^{\Lambda} \qquad \Lambda ::= (\widehat{vr}, t) \mid \epsilon$$

Reference Types

$$RT ::= UT \mid CT$$

$$CT ::= C \mid C\langle \eta \rangle \mid C\langle \_ \rangle$$

$$UT ::= U\langle R \rangle \mid U$$

Symbolic consent expression

Types and Type Environments

Data Types

Tag annotations record provenance and explicit tags

$$\mathrm{DT} ::= T^{\Lambda} \qquad \Lambda ::= (\widehat{vr}, t) \mid \epsilon$$

Reference Types

$$RT ::= UT \mid CT$$

$$CT ::= C \mid C\langle \eta \rangle \mid C\langle \_ \rangle$$

$$UT ::= U\langle R \rangle \mid U$$

Symbolic consent expression

Typing Environments

$\Gamma ::= \overline{vr} : DT$ Variables associated with data

$\Delta ::= vr : RT, \Delta \mid x : FT, \Delta \mid \epsilon$ Variables associated with objects, futures and users

Types and Type Environments

Data Types

Tag annotations record provenance and explicit tags

$$\mathrm{DT} ::= T^{\Lambda} \qquad \Lambda ::= (\widehat{vr}, t) \mid \epsilon$$

Reference Types

$$RT ::= UT \mid CT$$

$$CT ::= C \mid C\langle \eta \rangle \mid C\langle \_ \rangle$$

$$UT ::= U\langle R \rangle \mid U$$

Symbolic consent expression

Typing Environments

$\Gamma ::= \overline{vr} : DT$ Variables associated with data

$\Delta ::= vr : RT, \Delta \mid x : FT, \Delta \mid \epsilon$ Variables associated with objects, futures and users

Typing Judgements

$\Gamma; \Delta \vdash \alpha : ET$ Expressions

$\Gamma; \Delta \vdash s \triangleright \Gamma'; \Delta' \mid \mathbf{Cn}$ Statements

$\Gamma; \Delta \vdash sts \triangleright DT \mid \Gamma'; \Delta' \mid \mathbf{Cn}$ Sequence of statements

Core Data Model : Private Tags

```
data tag ⟨{alice}, {Healthcare, Psychotherapy}⟩
```

$\Rightarrow e \ \texttt{tag} \ t$

Core Data Model : Private Tags

```
data tag ⟨{alice}, {Healthcare, Psychotherapy}⟩
```

$\Rightarrow e \ \texttt{tag} \ t$

- Turns non personal data into personal data with owners and purposes

Core Data Model : Private Tags

```
data tag ⟨{alice}, {Healthcare, Psychotherapy}⟩
```

$\Rightarrow e \ \texttt{tag} \ t$

- Turns non personal data into personal data with owners and purposes

$$\frac{\Gamma;\Delta \vdash e : T^{\Lambda}}{\Gamma;\Delta \vdash e \ \textbf{tag} \ \langle \widehat{x}, \widehat{p} \rangle : T^{\Lambda \bullet (\emptyset, \langle \widehat{x}, \widehat{p} \rangle)}} \ (\textsc{T-E-TAG}) \quad x \in \widehat{x} \implies \Delta(x) = UT \wedge \textit{is-init}(UT)$$

Typing rule for tag expression

Core Data Model : Private Tags

```
data tag ⟨{alice}, {Healthcare, Psychotherapy}⟩
```

$\Rightarrow e \ \texttt{tag} \ t$

- Turns non personal data into personal data with owners and purposes

$$\frac{\Gamma;\Delta \vdash e : T^{\Lambda}}{\Gamma;\Delta \vdash e \ \textbf{tag} \ \langle \widehat{x}, \widehat{p} \rangle : T^{\Lambda \bullet (\emptyset, \langle \widehat{x}, \widehat{p} \rangle)}} \ (\textsc{T-E-TAG}) \quad x \in \widehat{x} \implies \Delta(x) = UT \wedge \mathit{is\text{-}init}(UT)$$

Typing rule for tag expression

- Combines tag annotation from subexpression
- Users in tags must be defined have user types
- Purposes are subset of program purposes

Constraint Generation

```
Alice -> ({Hospital}, {use, collect, store, transfer}, {Healthcare, Research})
```

Constraint Generation

```
Alice -> ({Hospital}, {use, collect, store, transfer}, {Healthcare, Research})
```

Each language construct that handles personal data is classified as one of four actions

$$x := \alpha \longrightarrow \textbf{use}, \textbf{collect}$$

$$\textbf{this}.x := \alpha \longrightarrow \textbf{use}, \textbf{collect}, \textbf{store}$$

$$vr!m(\overline{\alpha}) \longrightarrow \textbf{use}, \textbf{transfer}$$

$$\text{return } \alpha \longrightarrow \textbf{use}, \textbf{transfer}$$

Policies and consent are defined over these actions

Constraint Generation

```
Alice -> ({Hospital}, {use, collect, store, transfer}, {Healthcare, Research})
```

Each language construct that handles personal data is classified as one of four actions

$$x := \alpha \longrightarrow \textbf{use}, \textbf{collect}$$

$$\textbf{this}.x := \alpha \longrightarrow \textbf{use}, \textbf{collect}, \textbf{store}$$

$$vr!m(\overline{\alpha}) \longrightarrow \textbf{use}, \textbf{transfer}$$

$$\text{return } \alpha \longrightarrow \textbf{use}, \textbf{transfer}$$

Policies and consent are defined over these actions

- Constraints arise from handling statements referring to personal data

$A_{\text{Cnstr}}(act, \Lambda, \Delta)$ states which actions must be permitted (and are discharged against runtime state i.e., Consent

Core Runtime Context : Consent

Global user specific consent (Σ)

- Mutable policy state
- Evolves at runtime with (addCon and remCon) constructs
- Provides authorization on data operations

$$\Sigma ::= \emptyset \mid \Sigma,\, u \mapsto \chi \qquad \text{where} \qquad \chi ::= \emptyset \mid \chi,\, a \mapsto \langle \eta, p \rangle$$

Core Runtime Context : Consent

Global user specific consent (Σ)

- Mutable policy state
- Evolves at runtime with (addCon and remCon) constructs
- Provides authorization on data operations

$$\Sigma ::= \emptyset \mid \Sigma,\, u \mapsto \chi \qquad \text{where} \qquad \chi ::= \emptyset \mid \chi,\, a \mapsto \langle \eta, p \rangle$$

```
Alice -> ({Hospital}, {use, collect, store, transfer}, {Healthcare, Research})
```

Typing Rule : Field Assignment

$$\frac{\Gamma;\Delta \vdash \alpha : T^{\Lambda'}}{\Gamma, \mathbf{this}.x : T^{\Lambda};\Delta \vdash \mathbf{this}.x := \alpha \rhd \Gamma, \mathbf{this}.x : T^{\Lambda[\Lambda']};\Delta \,|\, Cnstr_S(\Lambda[\Lambda'],\Delta)} \; \Lambda'{=}\epsilon \vee \Lambda{\neq}\epsilon \quad (\textsc{T-ASGN-FIELD})$$

Typing Rule : Field Assignment

$$\frac{\Gamma;\Delta \vdash \alpha : T^{\Lambda'}}{\Gamma, \textbf{this}.x : T^{\Lambda};\Delta \vdash \textbf{this}.x := \alpha \rhd \Gamma, \textbf{this}.x : T^{\Lambda[\Lambda']};\Delta \,|\, Cnstr_S(\Lambda[\Lambda'],\Delta)} \; \Lambda'=\epsilon \vee \Lambda\neq\epsilon \quad (\text{T-ASGN-FIELD})$$

Type of the expression to be assigned is compatible with the one of the variable

Typing Rule : Field Assignment

$$\frac{\Gamma;\Delta \vdash \alpha : T^{\Lambda'}}{\Gamma, \textbf{this}.x : T^{\Lambda};\Delta \vdash \textbf{this}.x := \alpha \rhd \Gamma, \textbf{this}.x : T^{\Lambda[\Lambda']};\Delta \,|\, Cnstr_S(\Lambda[\Lambda'],\Delta)} \; (\text{T-ASGN-FIELD}) \quad \Lambda'{=}\epsilon \vee \Lambda{\neq}\epsilon$$

Type of the expression to be assigned is compatible with the one of the variable

Non-personal ⟵ Personal data

`String this.record;` $\Rightarrow \Lambda = \epsilon$

`String# p;` $\Rightarrow \Lambda' \neq \epsilon$

`this.record := p;` $\Rightarrow \Lambda_{new} = \epsilon[\Lambda']$

$$\Lambda[\Lambda'] = \begin{cases} \Lambda' & \text{if } \Lambda' \neq \epsilon \ \vee \ \Lambda = \epsilon \\ (\emptyset, \langle \emptyset, \mathcal{P} \rangle) & \text{otherwise} \end{cases}$$

Typing Rule : Field Assignment

$$\frac{\Gamma;\Delta \vdash \alpha : T^{\Lambda'}}{\Gamma, \textbf{this}.x : T^{\Lambda};\Delta \vdash \textbf{this}.x := \alpha \rhd \Gamma, \textbf{this}.x : T^{\Lambda[\Lambda']};\Delta \,|\, Cnstr_S(\Lambda[\Lambda'],\Delta)} \; \Lambda'{=}\epsilon \vee \Lambda{\neq}\epsilon \quad (\text{T-ASGN-FIELD})$$

Type of the expression to be assigned is compatible with the one of the variable

Personal ⟵ Non-personal data

```
String# this.record; ⇒ Λ ≠ ϵ
String p; ⇒ Λ′ = ϵ
this.record = p; ⇒ Λ_new = Λ[ϵ]
```

$$\Lambda[\Lambda'] = \begin{cases} \Lambda' & \text{if } \Lambda' \neq \epsilon \ \vee \ \Lambda = \epsilon \\ (\emptyset, \langle \emptyset, \mathcal{P} \rangle) & \text{otherwise} \end{cases}$$

Typing Rule : Method & Class

$$\frac{\forall M \in \overline{M}\ \exists \mathbf{Cn} \quad M, \mathbf{Cn}\ \textbf{OK in}\ C \qquad \forall A x \in \overline{A x}\ \ A \neq \mathsf{U}}{\textbf{class}\ C(\overline{A x})\ \{\ \overline{M}\ \}\ \textbf{OK}}\ (\textsc{T-CLASS})$$

$$\frac{\Gamma_{lc}\,\Gamma_{p\&f}; \Delta_{lc}\,\Delta_{p\&f}, \textbf{this} : C \vdash sts \rhd T^{\Lambda'} \mid \Gamma; \Delta \mid \mathbf{Cn}}{B\ m(\overline{A x}){:}\Lambda\{\ \overline{A' y};\ sts\}, \langle \mathbf{Cn}, \mathbf{Um} \rangle\ \textbf{OK in}\ C}\ (\textsc{T-METH})$$

$$\begin{array}{l} fields(C) = \overline{A''}\,\overline{z} \\ B\ m(\overline{A x}){:}\Lambda\{\ \overline{A' y};\ sts\} \in C \\ TagOk(\Lambda, \overline{A x}) \\ CmpTy(B, T^{\Lambda})\ \text{and}\ \Lambda \sqsubseteq \Lambda' \\ (\Gamma_{lc}, \Delta_{lc}) = TE_{lc}(\overline{A'}\,\overline{y}) \\ (\Gamma_{p\&f}, \Delta_{p\&f}) = TE_{p\&f}(\overline{A x}\,\overline{A''}\ \textbf{this}.\overline{z}) \\ \mathbf{Um} = \{x \mid \Delta(x) = \mathsf{U}\langle R \rangle \wedge R \neq \_ \wedge\ R \neq \emptyset\ \} \end{array}$$

Intuition:

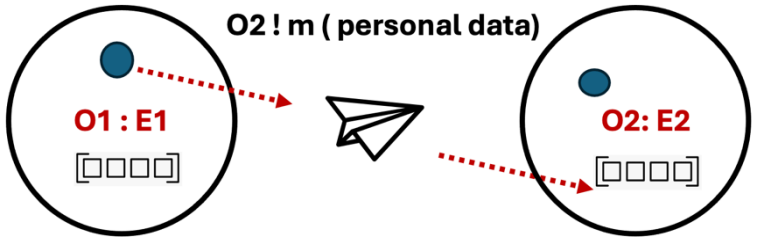
The method body is type-checked, and the method is annotated with
(i) the constraints it generates and
(ii) the user-consent variables it may affect.

Typed Operational Semantics

$cn\ \Sigma \rightarrow cn'\ \Sigma'$ & $o(\varphi, \gamma, q, \eta)^C$ can be either idle or running object

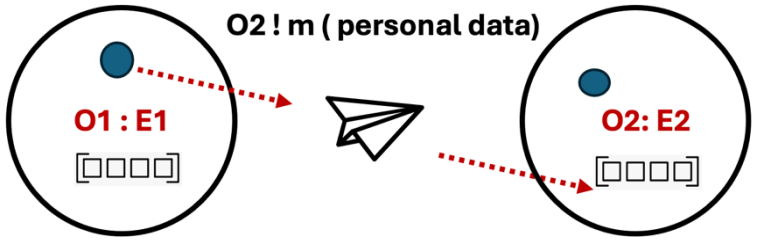
Typed Operational Semantics

$cn\ \Sigma \rightarrow cn'\ \Sigma'$ & $o(\varphi, \gamma, q, \eta)^C$ can be either idle or running object

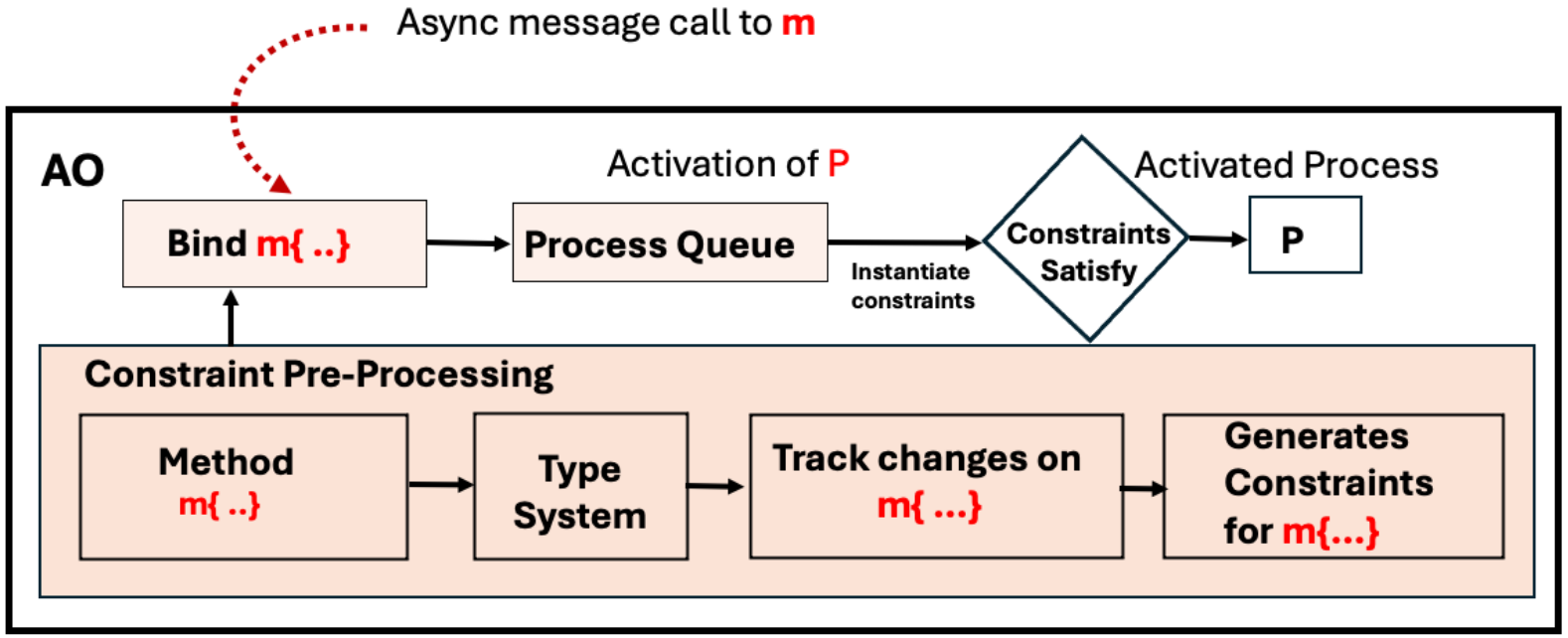


Typed Operational Semantics

$cn\ \Sigma \rightarrow cn'\ \Sigma'$ & $o(\varphi, \gamma, q, \eta)^C$ can be either idle or running object



- Rule bind method and activation are attached with constraints
- Rule activate checks constraints gathered by the type system
- Checks against the runtime environment



Typed Operational Semantics

$cn\ \Sigma \rightarrow cn'\ \Sigma'$ & $o(\varphi, \gamma, q, \eta)^C$ can be either idle or running object

(BIND-MTD)

$$\frac{I = \{i \mid val_i =_{t_i} v_i\} \quad \bigwedge_{i \in I} comply_T(\Sigma, \eta', t_i) \quad B\ m(\overline{A}\,\overline{x}){:}\Lambda\{\ \overline{A'}\,\overline{y};\ sts\}, \langle \mathbf{Cn}, \mathbf{Um} \rangle \text{ OK in } C}{o(\varphi, \gamma, q, \eta)^C\ m(o, o', \overline{val}, f, \eta')\ cn\ \Sigma \rightarrow o(\varphi, \gamma, q\ \langle \{\overline{x} \mapsto \overline{val}\ \overline{y} \mapsto \overline{def(A')} \mid sts\}^f_{\eta'}, \langle \mathbf{Cn}, \mathbf{Um} \rangle \rangle_m, \eta)^C\ cn\ \Sigma}$$

(ACTIVATE)

$$\frac{scheduler(q) = p = \langle \{\sigma \mid sts\}^f_{\eta'}, \langle \mathbf{Cn}, \mathbf{Um} \rangle \rangle \quad \forall \pi \in InstCnstr(\mathbf{Cn}, \langle \varphi, \sigma \rangle, \eta', \eta, \Sigma) \quad \pi \text{ holds} \quad Um = \{\sigma(x) \mid x \in \mathbf{Um}\}\ Uc = Ck(\Lambda, \langle \varphi, \sigma \rangle) \quad \forall o'(\ldots, \langle Um', Uc' \rangle, \ldots)^{C'} \in cn \quad \neg conflict(\langle Um, Uc \rangle, \langle Um', Uc' \rangle)}{o(\varphi, idle, q, \eta)^C\ cn\ \Sigma \rightarrow o(\varphi, \{\sigma \mid sts\}^f_{\eta'}, \langle Um, Uc \rangle, (q \setminus p), \eta)^C\ cn\ \Sigma}$$

Typed Operational Semantics

$cn\ \Sigma \rightarrow cn'\ \Sigma'$ & $o(\varphi, \gamma, q, \eta)^C$ can be either idle or running object

(BIND-MTD)

$$\frac{I = \{i \mid val_i = {}_{t_i} v_i\} \quad \bigwedge_{i \in I} comply_T(\Sigma, \eta', t_i) \qquad B\ m(\overline{A}\,\overline{x}){:}\Lambda\{\overline{A'}\,\overline{y};\ sts\}, \langle \mathbf{Cn}, \mathbf{Um} \rangle \text{ OK in } C}{o(\varphi, \gamma, q, \eta)^C\ m(o, o', \overline{val}, f, \eta')\ cn\ \Sigma \rightarrow o(\varphi, \gamma, q\ \langle \{\overline{x} \mapsto \overline{val}\ \overline{y} \mapsto \overline{def(A')} \mid sts\}^f_{\eta'}, \langle \mathbf{Cn}, \mathbf{Um} \rangle \rangle_m, \eta)^C\ cn\ \Sigma}$$

(ACTIVATE)

$$\frac{scheduler(q) = p = \langle \{\sigma | sts\}^f_{\eta'}, \langle \mathbf{Cn}, \mathbf{Um} \rangle \rangle \qquad \forall \pi \in InstCnstr(\mathbf{Cn}, \langle \varphi, \sigma \rangle, \eta', \eta, \Sigma) \quad \pi \text{ holds}}{\begin{array}{c} Um = \{\sigma(x) | x \in \mathbf{Um}\}\ Uc = Ck(\Lambda, \langle \varphi, \sigma \rangle) \\ \forall\, o'(\ldots, \langle Um', Uc' \rangle, \ldots)^{C'} \in cn \quad \neg conflict(\langle Um, Uc \rangle, \langle Um', Uc' \rangle) \end{array}}$$

$$o(\varphi, idle, q, \eta)^C\ cn\ \Sigma \rightarrow o(\varphi, \{\sigma | sts\}^f_{\eta'}, \langle Um, Uc \rangle, (q \setminus p), \eta)^C\ cn\ \Sigma$$

Typed Operational Semantics

$cn\ \Sigma \rightarrow cn'\ \Sigma'$ & $o(\varphi, \gamma, q, \eta)^C$ can be either idle or running object

Definition 1. $Ck(\Lambda, \langle \varphi, \sigma \rangle) = \hat{u}$ where $\langle \widehat{u}, \widehat{p} \rangle = InstAnn(\Lambda, \langle \varphi, \sigma \rangle)$

$$conflict(\langle U_m, U_c \rangle, \langle U'_m, U'_c \rangle) \iff (U_m \cap U'_m) \cup (U_m \cap U'_c) \cup (U'_m \cap U_c) \neq \emptyset$$

Typed Operational Semantics

$cn\ \Sigma \rightarrow cn'\ \Sigma'$ & $o(\varphi, \gamma, q, \eta)^C$ can be either idle or running object

Definition 1. $Ck(\Lambda, \langle \varphi, \sigma \rangle) = \hat{u}$ where $\langle \widehat{u}, \widehat{p} \rangle = InstAnn(\Lambda, \langle \varphi, \sigma \rangle)$

$$conflict(\langle U_m, U_c \rangle, \langle U'_m, U'_c \rangle) \iff (U_m \cap U'_m) \cup (U_m \cap U'_c) \cup (U'_m \cap U_c) \neq \emptyset$$

Activation Safety Guarantees

No consent interference

- A process activates only if no running process modifies or checks consent for the same users

Stable consent assumptions

- Consent conditions verified at activation remain valid during execution

Compliance through Soundness of TyPAOL

Soundness

If a program *PR*, is well typed, any non-idle object in a configuration is either waiting for a future or can make a reduction step

Compliance through Soundness of TyPAOL

Soundness

If a program *PR,* is well typed, any non-idle object in a configuration is either waiting for a future or can make a reduction step

- Running object will not get stuck for OO and other errors
- All privacy checks of handling personal data will always hold

Compliance through Soundness of TyPAOL

Soundness

If a program *PR*, is well typed, any non-idle object in a configuration is either waiting for a future or can make a reduction step

- Running object will not get stuck for OO and other errors
- All privacy checks of handling personal data will always hold

Tag consistency

- Tagged values evolve in accordance with the tag annotations of their variables.

Consent tracking

- User-consent changes required by a method are reflected in the policy expressions of the associated variables.

Compliance through Soundness of TyPAOL

Soundness

If a program *PR*, is well typed, any non-idle object in a configuration is either waiting for a future or can make a reduction step

Stability of Consent Assumptions

Consent constraints are validated at method activation

Soundness relies on:

- Alias control for user references
- Interference freedom between running objects

Key Summary

Goal : *Privacy-by-design via types:* ensure personal data is processed only under valid user consent

Key Summary

Goal : *Privacy-by-design via types:* ensure personal data is processed only under valid user consent

- Treat **privacy compliance as a typing problem**:
  - Types track personal data, users, and purposes
  - Typing infers which consent-dependent actions needed

Key Summary

Goal : *Privacy-by-design via types:* ensure personal data is processed only under valid user consent

- Treat **privacy compliance as a typing problem**:
  - Types track personal data, users, and purposes
  - Typing infers which consent-dependent actions needed

Hybrid Enforcement

- Type system tracks data–consent dependencies and generates consent constraints
- Check constraints against consent only at *method binding & activation*

Key Summary

Goal : *Privacy-by-design via types:* ensure personal data is processed only under valid user consent

- Treat **privacy compliance as a typing problem**:
  - Types track personal data, users, and purposes
  - Typing infers which consent-dependent actions needed

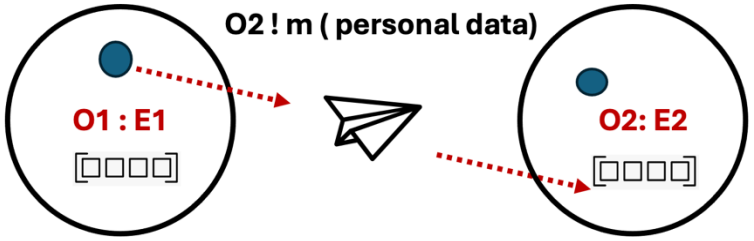
Hybrid Enforcement

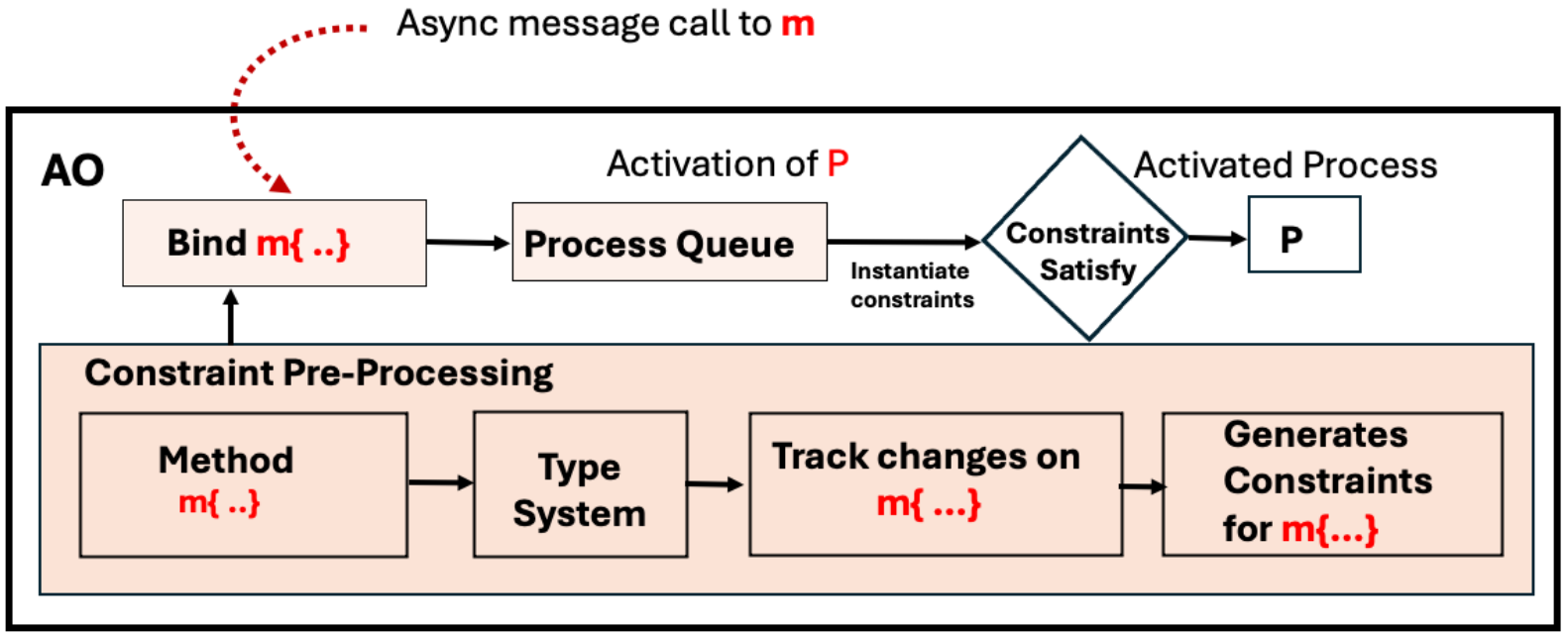
- Type system tracks data–consent dependencies and generates consent constraints
- Check constraints against consent only at *method binding & activation*

Guarantees

- No execution stops mid-method due to consent failure
- Tagged values evolve according to annotations

A Type System for Data Privacy Compliance in Active Object Languages





Questions?

Chinmayi Baramashetru
Research Associate, University of Kent
Email : C.Baramashetru@kent.ac.uk